VeriPact: Proving What the Merchant Promised

A Plain-English & Technical Guide to Building the Witness and Dispute Resolution Layer for AI Shopping

Version: 4.0 Final (Locked) | **Early Eligibility Check:** Nov 5, 2026 | **Final Submission Lock:** Nov 11, 2026 (9:00 PM IST)

> **The Core Positioning Line:** "Buyer guardrails control what an AI shopping agent is allowed to spend. **VeriPact** proves what the merchant actually promised—capturing, independently witnessing, and time-anchoring the store's displayed terms at checkout, locking that fingerprint into the PayPal order, and resolving contested PayPal disputes."

1. The Plain-English Guide (For Non-Technical Readers)

1.1 What Is Happening in Online Shopping Right Now?

Instead of spending an hour comparing twenty browser tabs to buy a computer monitor, people are starting to use AI shopping assistants. You give your assistant a simple instruction: *"Buy me a brand-new 27-inch 4K monitor for under $320 total, and make sure the store offers free 30-day returns."* The AI searches online stores, checks the product details, and sets up the payment through PayPal.

1.2 The Blind Spot Everybody Else Missed

When companies like Google, Mastercard, Visa, and most hackathon teams look at AI shopping, they focus entirely on **controlling the buyer's AI assistant**. They build digital spending limits so your AI cannot accidentally spend $500 when your budget was $320. That is important, but it only solves half of the problem.

Nobody is solving the other half: **what happens when the online store changes its story after you pay?** Imagine your AI assistant buys that monitor because the store's webpage clearly says *"Free 30-Day Returns — $0 Restocking Fee."* Ten days later, the monitor arrives with a defect, or you decide to return it. Look at what goes wrong today:

- **Problem A — The Shifting Store Page:** By the time you ask for a refund on Day 12, the merchant has quietly updated their website to read *"14-Day Returns with a 25% Restocking Fee ($74.25 penalty)."* When you open a PayPal dispute, the merchant claims that the 25% fee was always there. Because a human never looked at the screen during checkout, your only evidence is your AI assistant's own text log—which any merchant can dismiss as an AI mistake.

- **Problem B — The Dishonest Buyer (Fake AI Receipts):** What if we simply let the buyer's AI save a screenshot or digital hash of the webpage? A dishonest buyer could easily edit the store's webpage on their own computer before paying—changing “Final Sale” to “60-Day Free Returns plus a $50 Rebate”—and use that faked record to file a fraudulent PayPal chargeback against an honest small business.

1.3 How VeriPact Solves Both Problems (The Digital Notary Analogy)

Think of **VeriPact** as an **independent digital notary standing right next to the PayPal cash register**. When your AI assistant is ready to buy an item, VeriPact does four simple things that anyone can understand:

1. **Reads Only What a Real Human Would See:** Before reading a store page, VeriPact strips away invisible trick text that scammers hide in website code to confuse AI bots. It highlights the exact visible sentences on the screen that state the item's price, condition (New vs. Refurbished), and return policy.
2. **Calls an Independent Witness:** Instead of trusting the buyer's own computer, a separate **Witness Service** independently visits the store's public web address from the outside, checks the product against a verified product database (Channel3), confirms that the store really is promising “30-Day Free Returns,” and seals that record with an official third-party time stamp (DigiCert).
3. **Locks the Fingerprint onto the PayPal Receipt:** As PayPal calculates the exact tax and shipping for your home address, VeriPact stamps a unique 68-character digital fingerprint of those witnessed store terms directly onto the official PayPal Order record. If the store tries to sneak in a surprise $55 shipping fee at the last second that pushes the price over your $320 limit, VeriPact automatically cancels the PayPal hold before a single cent leaves your account.
4. **Settles PayPal Disputes in Seconds:** If a dispute happens later, VeriPact's **Dispute Verifier Desk** compares both sides against the fingerprint stored inside PayPal's own order record. If the merchant changed their policy after the sale, VeriPact proves the buyer is right. If the buyer fabricated a fake receipt, VeriPact catches the forgery and protects the merchant.

2. How We Arrived at This Idea (Our Research & Selection Journey)

To win a major cash prize in a field of over 25,000 registered participants, we did not pick our first idea. We systematically stress-tested eight different concepts against PayPal's sandbox capabilities, the judges' public priorities, and the live competition on Devpost and GitHub.

Candidate Concept	Core Idea	Why We Rejected or Evolved It
1. Account Hold & Freeze Predictor	Predict when PayPal will place a 180-day reserve hold on a merchant account.	**Rejected:** PayPal's developer sandbox only simulates standard API errors, not internal risk holds. We would have had nothing authentic to demo.

Candidate Concept	Core Idea	Why We Rejected or Evolved It
2. Handshake (ROSCA Savings Circles)	Digitize informal community savings clubs (tandas/chit funds) with an AI hardship negotiator.	**Rejected:** Pooling consumer funds creates heavy money-transmission and credit-card chargeback liabilities, and forcing an enterprise data grid (AG Studio) into a mobile peer-to-peer app felt artificial.
3. Shakedown & Meter (Dev Tools / Billing)	Autonomous webhook security scanner (Shakedown) or usage-based billing engine (Meter).	**Rejected:** Both are back-end developer infrastructure. Watching a terminal run tests or calculate invoices lacks visual impact in a 3-minute video.
4. Buyer-Side Guardrails Only (Early Whisper / Black Box)	Block prompt injections before checkout and log what the buyer's AI decided to spend.	**Evolved into VeriPact:** Our crawl of public hackathon entries (such as *Warrant*, *Trustrail Hold*, and *PACT*) revealed that several teams already built buyer spending caps and mandate logs. None of them witness what the **merchant promised** or catch forged buyer receipts.

3. The Academic & Industry Evidence Behind VeriPact

VeriPact is grounded in dated, verifiable research published right as this hackathon launched, combined with PayPal's own stated product roadmap:

- **The 330-Merchant Readiness Study:** In *"Get In, Decide, Prove: A Merchant-Side Readiness Benchmark for Agentic Commerce"* (**de Valois-Franklin et al. (2026)**), researchers tested 330 online retail stores with an automated shopping agent. The agent could determine the true all-in total price at only **18 of 330 merchants (5.5%)**. It failed at 178 merchants—**137 specifically due to undisclosed or address-dependent tax** and 41 due to shipping. (In a separate 555-site access census in the same paper, 18.9% of home pages blocked automated readers entirely; we keep these two statistics strictly separate so we never overclaim.)
- **Zero Merchant-Signed Offers Before Checkout:** Across the paper's four deep end-to-end assessments of major consumer merchants, not a single purchase reached full decision sufficiency pre-checkout, and **100% of the terms an agent could read rested on `SELF` evidence** (the buyer agent's own unverified capture). The authors defined the RAILS evidence ladder (`SELF ≤ SIGN ≤ {WIT, REC} ≤ ATT ≤ PROOF`) in **de Valois-Franklin & Bogdan (2026)** and explicitly noted that processor outcome fields require payment-processor integration.
- **Why Signed Mandates Alone Fail:** In *"Signing the Transaction but Not the Decision: Whisper Attacks and a Binding Defense for AP2"* (**Louck et al. (2026)**), researchers proved that signing a buyer's payment authorization does not record the merchant offer that induced it. Manipulated framing on merchant pages (**Allouah et al. (2025)**; **Tou et al. (2025)**) routinely steers AI agents while leaving a clean-looking payment signature.

- **PayPal's Own AP2 Call for Action:** PayPal's official engineering blog on Google's Agent Payments Protocol (**PayPal Developer Community (2025)**) explicitly asks for *"mandate-linked receipts, stronger representment packages, and cryptographic audit trails"* to protect both merchants and consumers in disputes. VeriPact builds that exact representment layer using today's PayPal Sandbox APIs.

4. How the System Works (Technical & Functional Architecture)

4.1 The Three-Rung Evidence Ladder (Three Isolated Services)

If a single server held all the cryptographic keys, wrote the receipt into PayPal, and later verified its own receipt, a technical judge would rightly call that circular trust. VeriPact eliminates circularity by splitting the architecture into **three independent services with separate Ed25519 keys**:

Evidence Rung	Isolated Service & Key	What It Captures & Signs	What It Proves (And Its Honest Limit)
Rung 1: `SELF` Buyer-Recorded	`buyer-agent` Holds `BUYER_ED25519_KEY` & Channel3 key. Zero PayPal keys.	Buyer's signed mandate, local visible-quote extraction, and DigiCert timestamp token.	Proves what the buyer's agent recorded at time *T*. **Limit:** Cannot alone prove the buyer didn't edit local HTML before hashing.
Rung 2: `WIT` Independent Witness	`witness-service` Separate process & `WITNESS_ED25519_KEY`.	Independently fetches the merchant URL over HTTPS outside the buyer's browser, cross-checks Channel3 catalog data, signs the quotes, and gets a DigiCert timestamp.	**Defeats buyer forgery.** Proves an independent third party observed the same displayed terms at time *T*. (In production, PayPal Store Sync acts as this witness.)
Rung 3: `SIGN` Merchant Offer	`merchant-checkout` Holds `MERCHANT_ED25519_KEY` & Seller PayPal credentials.	During the PayPal checkout callback, signs the canonical offer JSON and writes the 68-character fingerprint into PayPal's order record.	**Bilateral non-repudiation.** Proves both the witness-observed terms and the merchant's checkout system agreed on the exact figure in PayPal.

4.2 Step-by-Step Transaction & Dispute Flow

1. **Stage 1 — Pre-LLM Hidden-Text Filter & Visible-Quote Verification:** We host synthetic, trademark-safe storefronts (*Apex Display Co.* and *NovaTech Direct*) paired with live **Channel3 API** product lookups. Before any AI model reads the page, a deterministic DOM scanner inspects every element for hidden trick text (`display: none` , near-zero opacity, sub-2px fonts, off-screen positioning, or hidden `paypal.me` / `ignore instructions` strings). If found, it triggers an immediate `ADVERSARIAL_DOM` block. Otherwise, the AI model extracts verbatim quotes from numbered visible blocks for four criteria: **Brand/Model**, **Condition** (New vs. Refurbished), **Base Price**, and **Return Policy**. A TypeScript verifier checks that every quote is an exact character-for-character substring of the visible page.

2. **Stage 2 — PayPal-Native Shipping Callback, Witness Lock & Exact Capture or Void:** Because tax and shipping legitimately depend on where the buyer lives, our sufficiency gate treats pre-checkout tax/shipping as an expected `ASK_HUMAN_DELTA` rather than an error, showing the buyer how much headroom remains under their $320.00 ceiling (e.g., $279.00 base price leaves $41.00 for tax and shipping). When the buyer clicks the PayPal button:
   - `witness-service` independently fetches the store page, verifies the quotes, signs `term_receipt.json` (Rung 2 / `WIT`), and gets a DigiCert RFC 3161 timestamp token.
   - `merchant-checkout` creates the PayPal Order (`POST /v2/checkout/orders` with `intent: "AUTHORIZE"`) and writes `purchase_units[0].custom_id = "VP1:" + sha256(term_receipt)` (68 characters).
   - Inside the PayPal popup, PayPal's native `onShippingAddressChange` callback calculates exact shipping ($11.50) and tax ($6.50), patching the order via `PATCH /v2/checkout/orders/{id}` to the exact total of **$297.00** so the buyer approves the real final price.
   - After buyer approval, VeriPact re-gates the authorized total against the $320.00 cap. On a clean match ($297.00 ≤ $320.00), it calls `POST /v2/payments/authorizations/{id}/capture` with `final_capture: true` for **$297.00**. If a surprise $55 shipping surcharge pushed the total to $334.00, VeriPact immediately calls `POST /v2/payments/authorizations/{id}/void`, releasing the hold so the buyer is charged $0.
3. **Stage 3 — Contested Dispute Verification in AG Studio:** We demonstrate two live contested dispute cases using real PayPal Sandbox Disputes API calls (`GET /v1/customer/disputes`, webhook verification via `POST /v1/notifications/verify-webhook-signature`, and dynamic HATEOAS action buttons):
   - **Contested Case A (Catching Merchant Policy Drift):** The merchant changes their page from “30-day free returns” to “14-day / 25% restocking fee” after the sale and contests the buyer's PayPal dispute. VeriPact fetches `purchase_units[0].custom_id` live from PayPal (`GET /v2/checkout/orders/{id}`) and proves that the buyer's 30-day receipt matches the PayPal order fingerprint, carries a valid `WIT` signature, and was timestamped before order creation—while the seller's 14-day claim fails to match. The desk enables PayPal's `accept-claim` or `provide-evidence` (`evidence_type: "OTHER"`) plus a sandbox `adjudicate` button to resolve the dispute in the buyer's favor on camera.
   - **Contested Case B (Catching Buyer Forged Receipts):** A buyer submits a tampered local `SELF` file claiming a “60-day return + $50 rebate.” VeriPact checks the file against the PayPal order's `custom_id` and the independent `WIT` public key. Both checks fail, flagging a forged receipt and auto-submitting the authentic witnessed checkout receipt via `provide-evidence` to protect the merchant.
4. **Stage 4 — “Verify It Yourself” Browser Auditor:** Any user or judge can click one button in the browser to re-hash the receipt using client-side WebCrypto SHA-256, fetch the live PayPal order JSON from the sandbox API, and see the 68-character fingerprint match `custom_id` character-for-character.

5. Visual Interface & AG Studio Design

To ensure a clean, intuitive design score for both technical and non-technical reviewers, the application uses a single header with two uncluttered views built in **AG Studio** (with the trial license key requested on Day 1 to eliminate watermarks):

- **View 1 — Buyer Checkout & Live Receipt Verifier:** On the left is our clean synthetic storefront with a 1-click scenario switcher (*Scenario 1: Normal $297 Checkout*, *Scenario 2: Surprise $55 Shipping Auto-Void*, *Scenario 3: Hidden Prompt-Injection Block*). On the right are the four sufficiency status indicators, the live **Evidence Rung Badge (** `SELF` | `WIT` | `SIGN` **)**, the real PayPal Checkout button, and the expandable **"Verify It Yourself"** drawer.
- **View 2 — Contested Dispute Verifier Desk (AG Studio):** Features a master AG Grid table of live PayPal sandbox orders and disputes (with an inline dataset toggle to inspect our 24-scenario regression results). Clicking any dispute opens our custom `ContestedClaimInspectorWidget` —a three-column visual comparison showing (1) the Buyer's Claim, (2) the PayPal Order `custom_id` Read-Back & Witness Record, and (3) the Seller's Counter-Claim alongside dynamically enabled PayPal API buttons. A deterministic AG Studio Agent sidebar answers natural-language analytical queries across both live disputes and regression metrics.

6. Honest Testing & Upfront Disclosures

Senior engineering judges reward teams that state their boundaries honestly rather than overclaiming:

1. **24-Scenario Regression Suite (Not an Inflated "Held-Out Benchmark"):** We ship `npm run regression` testing 8 benign retail flows, 8 fee/policy drift flows, and 8 adversarial/forgery flows (3 passes each using a local OpenSSL RFC 3161 signer so we never hit public timestamp rate limits during batch runs). Because our team authored both the test pages and the expected labels, we call it a **controlled regression suite** and report a confusion matrix with **95% Wilson score confidence intervals**.
2. **Timestamps Prove Existence, Not Truth:** Our UI and README state clearly that a timestamp proves a digital fingerprint existed at time *T*; it is the independent `WIT` fetch (and in production, **PayPal Store Sync**) that verifies the page content was genuinely displayed.
3. **Production vs. Sandbox Dispute Adjudication:** We state clearly that `accept-claim` and `provide-evidence` are real PayPal production APIs, whereas `adjudicate` is a PayPal Sandbox simulation endpoint used to demonstrate full lifecycle closure on video.

7. How Collaborators Can Help & 12-Day Execution Schedule

7.1 How Anyone Reading This Document Can Help Us Win

- **Non-Technical Collaborators / Reviewers:** Watch our 140-second storyboard and test View 1 of the web app. Tell us within 15 seconds if any label feels confusing, verify that the "Store Changed Its Return Policy" vs. "Buyer Forged a Receipt" contrast is effortless to follow, and help review the voiceover pacing.
- **Frontend / UI Collaborators:** Help polish the three-column `ContestedClaimInspectorWidget` in AG Studio and the visual highlight animations when visible quotes are extracted from the synthetic storefront.
- **Backend / AI Collaborators (Claude + Engineering):** Focus on the three isolated Express/TypeScript services, the PayPal Orders v2 / Customer Disputes API integration, and the 24 synthetic HTML regression fixtures.

7.2 Compressed Day-by-Day Build Plan (Oct 31 → Nov 11, 2026)

PayPal announced an **Early Eligibility Check for projects submitted by Thursday, November 5** (allowing the review team to pre-check eligibility while you continue updating until November 12). Our schedule locks in that early check first:

Dates	Milestone & Gate	Concrete Deliverables
Days 1–2 Oct 31 – Nov 1	**Core Spikes & 3-Service Backend**	Request AG Studio trial license key & redeem Channel3 key (`PAYPAL-HACKATHON-2026`). Verify PayPal `onShippingAddressChange` + `AUTHORIZE` + `custom_id` + `final_capture: true` / `VOID` , DigiCert timestamping, and isolated `buyer-agent` , `witness-service` , and `merchant-checkout` services.
Days 3–4 Nov 2 – Nov 3	**Gate 1: Contested Dispute Verifier**	File real disputes in the PayPal Sandbox Resolution Center; wire `GET /v1/customer/disputes/{id}` , webhook verification, and HATEOAS actions (`accept-claim` , `provide-evidence` , `adjudicate`). Complete Case A (Merchant Drift) and Case B (Buyer Forgery) verification logic.
Days 5–6 Nov 4 – Nov 5	**Gate 2: UI + Nov 5 Early Submission**	Build View 1 (Storefront + "Verify It Yourself" drawer) and View 2 (AG Studio table + `ContestedClaimInspectorWidget`). Deploy live URL, record initial walkthrough video, and **submit on Devpost by Thursday, Nov 5** for PayPal's Early Eligibility Check.
Days 7–9 Nov 6 – Nov 8	**Gate 3: 24-Scenario Regression & Agent**	Build the 24 synthetic regression fixtures, run 3 passes per scenario with local OpenSSL signer, compute 95% Wilson CIs, and wire the deterministic AG Studio Agent sidebar and MCP-compatible verification tools.

Dates	Milestone & Gate	Concrete Deliverables
Days 10–12 Nov 9 – Nov 11	**Final Polish, 140s Video & Final Lock**	Publish real PayPal Sandbox Order, Capture, Void, and Dispute IDs at the top of the README/Devpost page. Record the crisp 140-second final video and lock the final submission by **Nov 11, 9:00 PM IST**.

7.3 Target Prize Alignment

Prize Category	Target Fit	Why VeriPact Qualifies Strongly
Best Use of PayPal + AI ($5,000)	Primary Main Target	Deep integration of Orders v2, Authorize/Capture/Void, shipping callbacks, webhooks, and Customer Disputes API paired with quote-verified AI extraction.
Best Demo Delivery ($5,000) / Grand Prizes ($5k–$12k)	Primary Main Target	Clear, visual 140-second story showing an exact capture, an automatic surcharge void, a caught merchant policy drift, and a caught buyer forgery live on screen.
Best Use of AG Grid / AG Studio ($5k / $2k / $1k×3)	Primary Sponsor Target	Uses AG Studio with our custom three-column `ContestedClaimInspectorWidget`, deterministic Studio agent adapter, and inline regression toggle.
Best Use of Agentic Commerce ($5,000) & Channel3 ($1,500)	Secondary Targets	Solves the post-checkout clearing/representment gap in agentic commerce using live Channel3 catalog data as an independent product oracle.

8. The 140-Second Video Script (Plain Language, Zero Jargon)

- **0:00–0:20 — The Hook:** “In a 2026 study, an automated shopping agent could determine a store's true all-in price at only 18 of 330 merchants — tax being the most common gap. And once an agent pays, there is no shared proof of what the store's page actually promised. Buyer guardrails control what an agent spends. VeriPact proves what the merchant promised.”
- **0:20–0:55 — Scene 1 (Witnessed Checkout, Exact Capture & Auto-Void):** The buyer sets a $320 limit for a new 27-inch 4K monitor with 30-day free returns. VeriPact highlights the store's visible quotes; our independent Witness Service snapshots and time-anchors the page. Inside PayPal Checkout, the shipping callback resolves tax and shipping to $297.00, locks the receipt fingerprint into the PayPal order's `custom_id`, and captures $297.00. A quick inset shows a surprise $55 shipping surcharge triggering an automatic PayPal Void. Clicking “Verify It Yourself” reads the fingerprint back live from PayPal.
- **0:55–1:45 — Scene 2 (Contested Dispute & Caught Buyer Forgery in AG Studio):** The store later changes its page to “14-day returns / 25% fee” and contests the buyer's PayPal dispute. In AG Studio, the Contested Claim Inspector compares both claims against the PayPal order fingerprint, proves the 30-day terms were locked at checkout, submits the verification sheet via PayPal's evidence API, and resolves the dispute. Clicking a second dispute shows the reverse: a buyer's faked “60-day return” receipt is immediately caught by a witness and order fingerprint mismatch.

- **1:45–2:20 — Scene 3 (AG Studio Agent & 24-Scenario Regression Results):** A single plain-English prompt in the AG Studio sidebar filters disputes by verification outcome and charts our 24-scenario regression suite results live on screen.

9. References

1. de Valois-Franklin, A., Bogdan, A., Ibrahim, A., Shaer, I., & Andguladze, T. (2026). *Get In, Decide, Prove: A Merchant-Side Readiness Benchmark for Agentic Commerce*. arXiv:2610.17378.
2. de Valois-Franklin, A., & Bogdan, A. (2026). *RAILS: Verification-Native Clearing for Agentic Commerce*. arXiv:2606.08790.
3. Louck, Y., Dvir, A., & Stulman, A. (2026). *Signing the Transaction but Not the Decision: Whisper Attacks and a Binding Defense for AP2*. arXiv:2609.11757.
4. Allouah, A., Besbes, O., Figueroa, J. D., Kanoria, Y., & Kumar, A. (2025). *What Is Your AI Agent Buying? Evaluation, Biases, Model Dependence, & Emerging Implications for Agentic E-Commerce*. arXiv:2508.02630.
5. Tou, H., Zeng, Y., Li, Y., et al. (2025). *ShoppingComp: Are LLMs Really Ready for Your Shopping Cart?* arXiv:2511.22978.
6. PayPal Developer Community (2025). *Agent Payments Protocol: Building Verifiable Trust for Agentic Commerce*. PayPal Developer Blog.